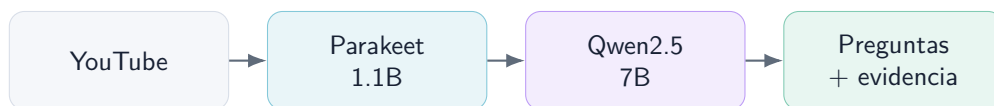


Youtube2knowledge

De video a conocimiento verificable

Arquitectura, modelos locales, resolución de incidentes y despliegue



Alberto Muñoz

NVIDIA DGX Spark GB10

2 de septiembre de 2026

youtube2knowledge-five.vercel.app

github.com/LuisAlbertoMunozUbando/youtube2knowledge

Resumen ejecutivo

Youtube2knowledge convierte un video público de YouTube en un paquete de conocimiento trazable. El sistema extrae y normaliza el audio, lo transcribe con NVIDIA Parakeet 1.1B RNNT Multilingual, usa Qwen2.5-7B-Instruct para formular preguntas educativas y respuestas, valida cada evidencia contra la transcripción y archiva el resultado completo en Google Drive.

La solución separa la experiencia web de la carga de cómputo. Next.js se publica en Vercel; Cloudflare Tunnel conecta el navegador con FastAPI en una NVIDIA DGX Spark; los modelos, las claves y los datos persistentes permanecen locales.

El desarrollo exigió resolver autenticación NGC, selección del perfil GB10, permisos del caché NIM, DNS dentro del contenedor, descarga Xet, parseo de CORS, formato de audio, alineación de evidencia, sincronización de Drive y dos fallos independientes de Vercel. El documento conserva la arquitectura final y el razonamiento diagnóstico que permitió llegar a ella.

DESPLIEGUE WEB EXITOSO

ASR VALIDADO

LLM VALIDADO

ARCHIVO DRIVE

Contenido

Resumen ejecutivo	i
1 Visión del producto	1
1.1 Principios	1
2 Arquitectura	1
2.1 Estados del pipeline	1
3 Plataforma NVIDIA DGX Spark	1
4 Pipeline de conocimiento	2
4.1 Ingesta y audio	2
4.2 Transcripción	2
4.3 Preguntas y grounding	2
4.4 Archivo durable	2
5 Problemas resueltos	3
5.1 Perfil y caché NIM	4
5.2 Red y pesos de Qwen	4
5.3 Configuración CORS	4
5.4 Evidencia con ASR ruidoso	4
5.5 Dos fallos distintos de Vercel	4
5.6 Ventana 502 durante la recreación de FastAPI	5
5.7 Dominio propio y restauración del túnel	5
6 Resiliencia	5
7 Validación de extremo a extremo	5
7.1 Infraestructura y modelos	5
7.2 Video real y Drive	6
7.3 Frontend	6
7.4 Aceptación pública completa	6
8 Seguridad y operación	6
9 Lecciones de ingeniería	6
10 Estado y siguientes pasos	7

11 Reproducibilidad	7
12 Fuentes técnicas	7
A Arquitectura local de modelos	8
B Anatomía de Qwen2.5-7B-Instruct	8
C Distribución de parámetros y pesos	8
D Memoria durante la inferencia	10
E Estado de validación	10
F Fuentes técnicas	10

1 Visión del producto

El video es sencillo de consumir, pero difícil de consultar, verificar y reutilizar. Una transcripción aislada tampoco basta: el usuario necesita preguntas relevantes, respuestas breves y un camino de regreso a la evidencia.

Youtube2knowledge permite seleccionar *What, Which, Where, When, How, Why, Who* y *Whose*, añadir palabras clave, escribir preguntas propias, elegir el idioma y fijar cuántas preguntas se generan por tipo.

1.1 Principios

- **Evidencia antes que elocuencia:** es mejor omitir que inventar.
- **IA local:** ASR y LLM se ejecutan en la DGX Spark.
- **Responsabilidades separadas:** web, edge, API, modelos y archivo.
- **Recuperación incremental:** un fallo del LLM no repite el ASR.
- **Estado durable:** jobs y resultados sobreviven recargas y reinicios.

2 Arquitectura

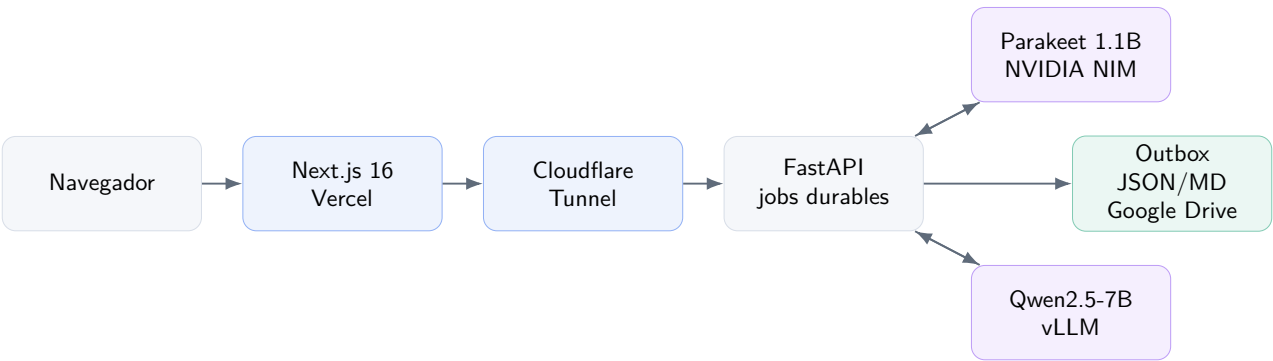


Figura 1: Arquitectura de producción: web pública y cómputo local desacoplados.

Frontera	Expuesto	Protegido
Vercel	HTML, CSS y JavaScript	Ninguna clave de modelo
Cloudflare	Host HTTPS de la API	IP y puertos internos
FastAPI	Salud y jobs	Validación, límites y estado
DGX Spark	Puerto 8020 por túnel	ASR 9000 y LLM 8001 en loopback
Google Drive	Carpeta de resultados	rclone montado sólo lectura

2.1 Estados del pipeline

3 Plataforma NVIDIA DGX Spark

La máquina objetivo usa arquitectura `aarch64`, GPU NVIDIA GB10, driver 580.159.03, CUDA 13.0 y Docker Compose 5.0.2. Durante la puesta en marcha había 121 GiB de memoria y 2.7 TiB libres.

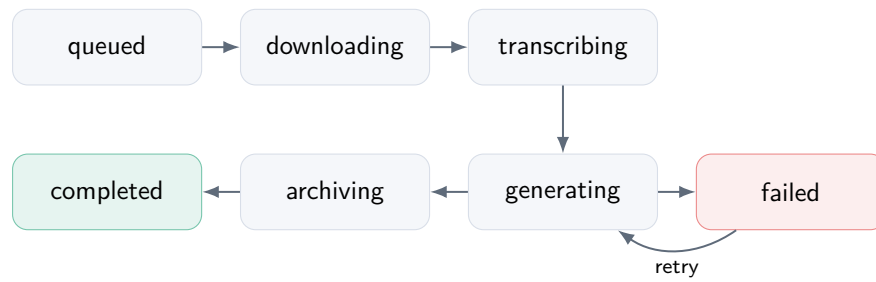


Figura 2: Máquina de estados persistente y reintento sólo de generación.

Tabla 1: Servicios del despliegue Spark.

Servicio	Puerto host	Cómputo	Función
asr	127.0.0.1:9000	GPU	Speech NIM con Parakeet
llm	127.0.0.1:8001	GPU	vLLM con Qwen2.5-7B
api	0.0.0.0:8020	CPU	FastAPI y jobs
drive-sync	sin puerto	CPU	Copia mediante rclone

4 Pipeline de conocimiento

4.1 Ingesta y audio

La API sólo acepta videos individuales de `youtube.com` y `youtu.be`. `yt-dlp` obtiene metadatos y el mejor audio. FFmpeg normaliza a WAV PCM de 16 bits, mono y 16 kHz:

```
ffmpeg -i source -vn -ac 1 -ar 16000 -c:a pcm_s16le audio.wav
```

Los audios grandes se dividen para mantener cada petición por debajo de 20 MiB.

4.2 Transcripción

Parakeet usa el perfil offline multilingüe de DGX Spark con VAD Silero y diarización Sortformer. La API envía `language=multi` para detección automática y habilita puntuación.

4.3 Preguntas y grounding

Qwen recibe tipos, preguntas propias, palabras clave, idioma y límite por tipo. Debe devolver JSON con `type`, `question`, `answer` y `evidence`. La evidencia se normaliza, alinea contra ventanas de tokens y, si supera umbrales conservadores, se reemplaza por el fragmento exacto de la transcripción. Si no hay sustento, la pregunta se descarta.

4.4 Archivo durable

Cada job completo escribe atómicamente JSON y Markdown en `/data/drive-outbox`. Ambos incluyen URL, metadatos, solicitud, transcripción, preguntas, respuestas y evidencia. rclone copia después a `knowledge-drive:AnswersFromYoutubeVideos`; una caída de Drive no impide terminar el job.

5 Problemas resueltos

Tabla 2: Incidentes, causas y correcciones.

Componente	Síntoma	Diagnóstico	Corrección
NGC	<code>unauthorized</code>	Clave inicial ausente o inválida	Login por stdin con usuario literal <code>\$oauth_token</code> ; validación exitosa.
Perfil NIM	No había perfil coincidente	El selector automático no fijaba GB10 y modo offline	Etiquetas exactas <code>gpu=dgx_spark, 2e12:10de, FP16, multilingüe, Silero y Sortformer</code> .
Caché NIM	<code>Permission denied</code>	Volumen persistente no escribible	Inicializador root de una ejecución antes del ASR.
Compose	<code>chmod: missing operand</code>	Entrypoint y command se combinaban mal	<code>entrypoint: ["chmod"]</code> más argumentos separados.
DNS LLM	Hugging Face resolvía a 0.0.0.0	DNS de contenedor distinto al host	DNS público limitado al servicio llm.
Descarga Qwen	Error CAS/Xet	Reconstrucción Xet inestable	<code>HF_HUB_DISABLE_XET=1</code> y descarga HTTP con timeout de 120 s.
FastAPI	Reinicio continuo por CORS	Pydantic intentaba decodificar la lista como JSON	NoDecode y validador separado por comas.
Raíz API	El dominio mostraba 404 Not Found	La API sólo definía <code>/healthz</code> y rutas de jobs	Endpoint <code>/</code> con estado, salud y enlace a la aplicación.
Dominio web	Vercel mostraba <i>Invalid Configuration</i>	Faltaba el CNAME específico del proyecto	<code>youtube2knowledge</code> a Vercel DNS, sin proxy de Cloudflare.
CORS dominio	El navegador mostraba <code>Failed to fetch</code>	El nuevo origen no estaba en la lista de FastAPI	Se añadieron ambos dominios web y localhost a <code>CORS_ORIGINS</code> .
Ruta API	El puerto 443 dejó de conectar	Se eliminó el hostname publicado del túnel	Una ruta única a <code>http://localhost:8020</code> restauró el servicio.
Redeploy API	Cloudflare devolvió HTTP 502 inmediatamente después de recrear	Uvicorn aún no escuchaba aunque Compose ya había terminado	Espera activa de <code>/healthz</code> ; no reiniciar túnel ni modelos.
Speech NIM	HTTP 400 con el video real	Audio en formato no aceptado	WAV PCM S16LE mono 16 kHz antes del envío.
Grounding	Evidencia no literal	Qwen reparaba errores fonéticos del ASR	Alineación difusa restringida y recuperación del texto exacto.
Drive	<code>directory not found</code>	La carpeta no existía	<code>rclone mkdir</code> y reinicio de <code>drive-sync</code> .
Vercel TS	Fallo en <code>Running TypeScript</code>	Validador interno duplicado sin diagnóstico	<code>tsc -noEmit</code> explícito y omisión sólo de la segunda validación.

Componente	Síntoma	Diagnóstico	Corrección
Vercel output	Faltaba el manifiesto NFT del servidor Next.js	<code>output: standalone</code> para autoalojamiento	era Eliminación de standalone y salida nativa de Vercel.

5.1 Perfil y caché NIM

La imagen era accesible y compatible, pero el selector no eligió la variante de Spark. Se inspeccionó el manifiesto y se fijó:

```
gpu=dgx_spark,gpu_device=2e12:10de,mode=ofl,
model_type=prebuilt,type=default,vad=silero,
diarizer=sortformer
```

Después apareció `os error 13`: red y autenticación funcionaban, pero el volumen no era escribible. Un inicializador root ajustó permisos. La primera forma del comando duplicaba el entypoint de la imagen; al separar ejecutable y argumentos terminó con código cero.

5.2 Red y pesos de Qwen

El host resolvía Hugging Face a direcciones públicas y recibía HTTP 200; dentro del contenedor el nombre resolvía a 0.0.0.0. Se aplicó DNS público sólo a 11m. Una segunda falla, independiente, ocurrió en CAS/Xet. Deshabilitar Xet activó la ruta HTTP tradicional y permitió cargar aproximadamente 15.23 GB decimales de pesos BF16.

5.3 Configuración CORS

`pydantic-settings` intentaba interpretar `list[str]` como JSON antes del validador. La anotación `NoDecode` permitió aceptar la forma operativa:

```
cors_origins: Annotated[list[str], NoDecode]

@field_validator("cors_origins", mode="before")
def split_origins(value):
    return [item.strip() for item in value.split(",") if item.strip()]
```

5.4 Evidencia con ASR ruidoso

El video real contenía terminología como DLSS. La transcripción produjo variantes fonéticas; Qwen devolvió una cita semánticamente correcta pero ortográficamente reparada. Aceptar cualquier paráfrasis habría debilitado el grounding.

La solución: tokenizar, proponer ventanas por tokens compartidos, medir similitud de caracteres y tokens, aplicar umbrales y devolver siempre el fragmento exacto de la transcripción. Las ventanas insuficientes se descartan.

5.5 Dos fallos distintos de Vercel

El primero ocurrió después de compilar, durante la validación integrada de tipos. El build pasó al separar `tsc -noEmit` y evitar repetir esa validación.

El segundo ocurrió en `onBuildComplete`: Vercel buscaba `.next/next-server.js.nft.json`. La opción `standalone` produce una distribución mínima para autoalojamiento; este frontend usa Next.js administrado por Vercel. Al retirar la opción, el commit final obtuvo estado `success`.

5.6 Ventana 502 durante la recreación de FastAPI

Tras desplegar el endpoint raíz se recreó únicamente el servicio `api`. Compose informó que el contenedor estaba creado, pero esa condición no implica que Uvicorn ya acepte conexiones. La primera solicitud pública llegó durante esa ventana y Cloudflare respondió 502 porque el origen `localhost:8020` todavía no escuchaba.

La espera activa de `http://127.0.0.1:8020/healthz` confirmó después `LOCAL_API_READY`; el contenedor apareció `healthy` y tanto `/` como `/healthz` devolvieron HTTP/2 200 a través de Cloudflare. No fue necesario reiniciar el túnel, Parakeet ni Qwen.

5.7 Dominio propio y restauración del túnel

El frontend funcionaba en el hostname técnico de Vercel, pero el dominio propio mostraba configuración inválida. Se añadió `youtube2knowledge.albertomunoz.ai` al proyecto y se creó en Cloudflare el CNAME exacto indicado por Vercel con proxy deshabilitado. El backend conservó un hostname independiente: `youtube2knowledge-api.albertomunoz.ai`.

La primera carga desde el dominio nuevo produjo `Failed to fetch`. FastAPI todavía no permitía ese origen y, durante la corrección, también se eliminó por accidente la ruta publicada de la API. El diagnóstico separó ambos fallos: CORS se corrigió en `.env`, mientras que el hostname se restauró en el túnel existente como una sola aplicación publicada hacia `http://localhost:8020`. No se creó otro túnel ni se expusieron los puertos internos de los modelos.

6 Resiliencia

- Los jobs se guardan como JSON atómico mediante temporal y `os.replace`.
- El navegador conserva el último ID en `localStorage`.
- El polling continúa cada 2.5 segundos tras errores transitorios.
- `POST /api/v1/jobs/{id}/retry-generation` reutiliza la transcripción y reinicia en 75 por ciento.
- Un semáforo limita la concurrencia; en Spark se usa un job por defecto.
- El outbox local desacopla el resultado de la disponibilidad de Drive.

7 Validación de extremo a extremo

7.1 Infraestructura y modelos

- CUDA 13.0 y GB10 visibles dentro de un contenedor.
- Login NGC y manifiesto de Parakeet accesibles.
- Speech NIM transcribió su muestra como *What is natural language processing?*
- vLLM informó `Application startup complete` y `/health` devolvió 200.
- Qwen generó JSON válido con preguntas What, Why y How.

- FastAPI quedó saludable en `/healthz`.

7.2 Video real y Drive

La prueba funcional utilizó youtu.be/58FagrSqC4M. El ejercicio descubrió los defectos de formato de audio y alineación de evidencia; ambos quedaron incorporados al código. El sistema generó JSON y Markdown en el outbox y, tras crear `AnswersFromYoutubeVideos`, los sincronizó a Google Drive.

7.3 Frontend

Vercel quedó conectado al repositorio con raíz `apps/web`. El despliegue corregido obtuvo estado exitoso y la aplicación quedó disponible en youtube2knowledge-five.vercel.app.

Como control operativo continuo, la aceptación completa debe incluir un preflight CORS desde ese origen y un job real que termine en Drive. Así se detectan cambios futuros de dominio, túnel o allowlist.

La API pública quedó validada en youtube2knowledge-api.albertomunoz.ai: la raíz descriptiva y `/healthz` respondieron HTTP/2 200 detrás de Cloudflare.

El dominio final de usuario quedó disponible en youtube2knowledge.albertomunoz.ai. El preflight de `POST /api/v1/jobs` respondió 200 y devolvió exactamente ese origen en la cabecera CORS de respuesta.

7.4 Aceptación pública completa

Desde el dominio público se creó el job `a6abd7dc33bb2d68` con el video de DLSS 5. Descarga, audio y ASR produjeron 11,278 caracteres de transcripción. La primera generación no conservó preguntas con evidencia suficiente y terminó de forma segura en `failed; retry-generation` reutilizó el ASR, alineó una cita exacta, completó el job al 100 por ciento y generó un paquete JSON y un informe Markdown para Drive.

8 Seguridad y operación

- Las claves NGC, Hugging Face y rclone permanecen en Spark.
- Los puertos de ASR y LLM se ligan a 127.0.0.1.
- Cloudflare Tunnel evita abrir puertos entrantes en el router.
- CORS usa orígenes exactos y rclone se monta sólo lectura.
- La API restringe hosts, videos individuales y duración.
- Se recomienda rate limiting para `POST /api/v1/jobs`.
- Sólo deben procesarse videos cuyo uso esté autorizado.

9 Lecciones de ingeniería

1. Separar autenticación, red, compatibilidad y permisos.
2. Probar conectividad dentro del contenedor, no sólo en el host.

3. Fijar explícitamente el perfil de hardware en imágenes multi-GPU.
4. Normalizar datos en la frontera antes de invocar un modelo.
5. Tolerar ruido del ASR sin aceptar evidencia inventada.
6. Desacoplar persistencia local de servicios externos.
7. Distinguir compilación, TypeScript y post-build en Vercel.
8. Usar la salida nativa del proveedor cuando éste administra el runtime.

10 Estado y siguientes pasos

Capacidad	Estado	Evidencia
ASR local	LISTO	NIM ready y muestra correcta
LLM local	LISTO	vLLM healthy y JSON válido
Grounding	LISTO	alineación y pruebas
Archivo Drive	LISTO	JSON y Markdown sincronizados
Frontend Vercel	LISTO	despliegue success
Dominio propio	LISTO	HTML y TLS HTTP/2 200
API Cloudflare	LISTO	raíz y salud HTTP/2 200
Flujo público	LISTO	job real completado y archivado
Rate limiting	SIGUIENTE	regla Cloudflare
Observabilidad	SIGUIENTE	métricas por etapa

Las mejoras naturales son autenticación opcional, cola durable para múltiples usuarios, búsqueda sobre el histórico, enlaces con timestamp a la evidencia y métricas de latencia y fallo.

11 Reproducibilidad

El repositorio público se encuentra en:

github.com/LuisAlbertoMunozUbando/youtube2knowledge

```
cp .env.spark.example .env
docker login nvcr.io --username '$oauthtoken'
docker compose --env-file .env \
  -f docker-compose.yml \
  -f docker-compose.spark.yml \
  config --quiet
docker compose --env-file .env \
  -f docker-compose.yml \
  -f docker-compose.spark.yml \
  up -d --build
```

La configuración poblada, los tokens y `rclone.conf` no forman parte del repositorio.

12 Fuentes técnicas

1. NVIDIA, [Parakeet 1.1B RNNT Multilingual ASR](#).

2. Qwen Team, [Qwen2.5-7B-Instruct](#).
3. vLLM, [documentación oficial](#).
4. Next.js, [Output File Tracing y standalone](#).
5. Vercel, [Next.js on Vercel](#).
6. Cloudflare, [Cloudflare Tunnel](#).

A Arquitectura local de modelos

Youtube2knowledge separa el procesamiento en dos redes especializadas. NVIDIA Parakeet RNNT Multilingual convierte audio en texto; Qwen2.5-7B-Instruct transforma la transcripción en preguntas y respuestas. Después de descargar los artefactos, ambas inferencias permanecen en la DGX Spark.

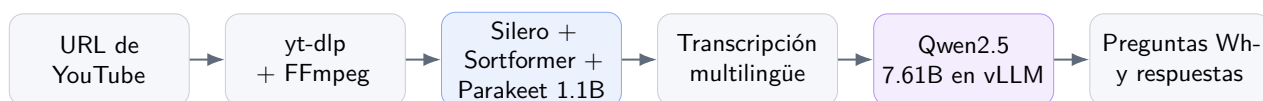


Figura 3: Pipeline local desde el video hasta el material de estudio.

Tabla 3: Modelos principales desplegados en la GB10.

Etapas	Modelo	Parámetros	Precisión	Responsabilidad
Voz a texto	Parakeet RNNT Multilingual	1.1B	FP16	Reconocimiento de voz en 25 idiomas y variantes.
Texto a conocimiento	Qwen2.5-7B-Instruct	7.61B	BF16	Generación estructurada de preguntas Wh- y respuestas.

B Anatomía de Qwen2.5-7B-Instruct

Qwen2.5-7B-Instruct es un Transformer *decoder-only*. Su configuración publicada especifica 28 bloques, dimensión oculta de 3,584, dimensión intermedia de 18,944, 28 cabezas de consulta, cuatro cabezas Key/Value y vocabulario de 152,064 tokens. La variante cargada por vLLM usa BF16 y un contexto operativo de 32,768 tokens.

La atención relaciona posiciones de la transcripción. La MLP transforma cada posición de manera no lineal y concentra la mayor parte de los parámetros. El conocimiento no está guardado como registros independientes: emerge de patrones distribuidos entre estas matrices.

C Distribución de parámetros y pesos

Cada parámetro BF16 ocupa dos bytes. Por ello, 7.6156 mil millones de parámetros requieren cerca de 15.23 GB decimales, o 14.19 GiB, antes de activaciones y cachés de inferencia.

Un bloque contiene alrededor de 29.4M parámetros de atención y 203.7M de MLP, para un total cercano a 233.1M. Los 28 bloques reúnen aproximadamente 6.53B parámetros. La tabla de embeddings y la cabeza de salida no comparten pesos en esta configuración.

Un bloque Transformer, repetido 28 veces

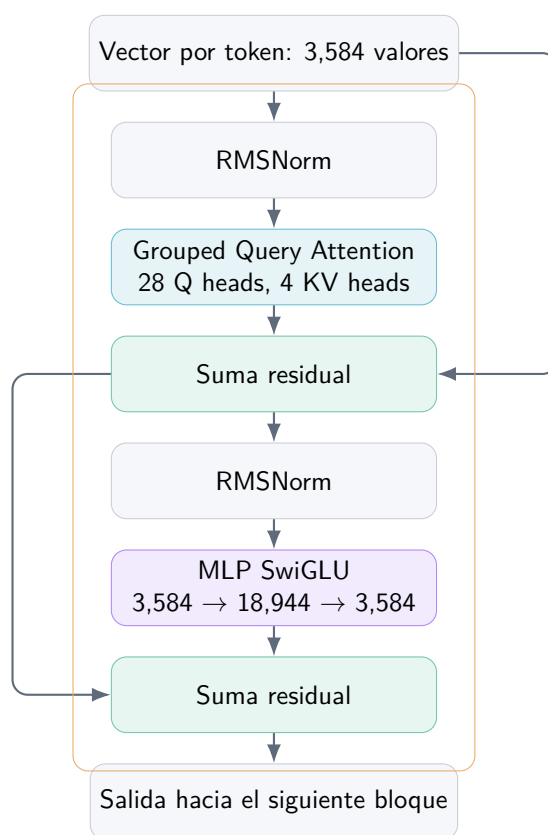


Figura 4: Flujo interno simplificado de un bloque Qwen2.5.

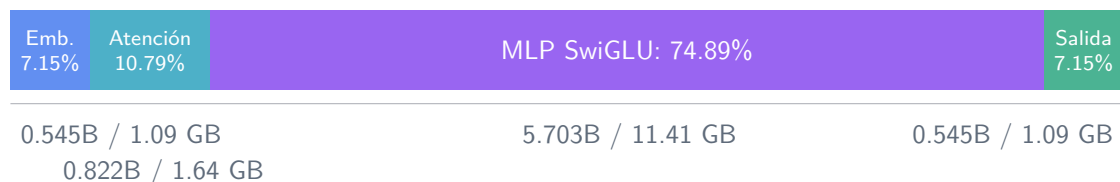


Figura 5: Distribución aproximada de los 7.61B parámetros de Qwen2.5-7B-Instruct.

Tabla 4: Cálculo de parámetros principales.

Componente	Forma conceptual	Parámetros	BF16 aprox.
Embedding	$152,064 \times 3,584$	0.545B	1.09 GB
Atención, 28 capas	Q, K, V y proyección de salida	0.822B	1.64 GB
MLP, 28 capas	Gate, Up y Down	5.703B	11.41 GB
Cabeza de salida	$3,584 \times 152,064$	0.545B	1.09 GB
Total	Incluye normas y sesgos menores	7.616B	15.23 GB

D Memoria durante la inferencia

Los pesos son sólo una parte del uso de memoria. vLLM también mantiene activaciones temporales, áreas de trabajo CUDA, grafos compilados y la caché KV. Esta última evita recalcular Keys y Values de todos los tokens anteriores cuando el modelo genera el siguiente token.

Con cuatro cabezas KV, dimensión de cabeza 128, 28 capas y BF16, el coste ideal por token es aproximadamente:

$$2 (K,V) \times 4 \text{ cabezas} \times 128 \times 28 \text{ capas} \times 2 \text{ bytes} = 57,344 \text{ bytes/token.} \quad (1)$$

Una secuencia de 32,768 tokens requiere cerca de 1.88 GB decimales de KV antes de considerar concurrencia y alineamientos internos. Grouped Query Attention reduce esta cifra porque 28 cabezas Query comparten sólo cuatro grupos KV.



Figura 6: Categorías de memoria del servicio LLM. El límite es un techo, no un consumo constante.

E Estado de validación

La cadena de modelos alcanzó los siguientes hitos en la DGX Spark GB10:

- Parakeet respondió correctamente a una transcripción de muestra por la API HTTP de NVIDIA NIM.
- El perfil NIM seleccionado corresponde a DGX Spark, FP16, modo offline multilingüe, VAD Silero y diarización Sortformer.
- Qwen2.5-7B-Instruct se cargó en BF16 con FlashAttention 2 mediante vLLM 0.28.0.
- El servidor vLLM informó `Application startup complete` y respondió 200 OK en `/health`.
- Los puertos de ASR y LLM permanecen enlazados a loopback; FastAPI actúa como orquestador local.

F Fuentes técnicas

NVIDIA, [Parakeet 1.1B RNNT Multilingual ASR Model Card](#); Qwen Team, [Qwen2.5-7B-Instruct Model Card](#); y Qwen Team, [Qwen2.5-7B-Instruct configuration](#).